

Capítulo 5 — GUÍA DE LECTURA

Capítulo 5 — GUÍA DE LECTURA.....	1
Asincronía y red.....	3
Promesas, <code>fetch</code> , errores y eventos del servidor, explicados en criollo	3
Antes de empezar: cómo usar esta guía.....	3
5.1 — De qué se trata esta clase	3
Qué dice	3
En criollo.....	3
5.2 — Por qué existe la petición en segundo plano	4
Qué dice	4
En criollo: la génesis, en tres movimientos.....	4
El problema siguiente fue de forma, no de capacidad	4
Las cuatro decisiones que explican todo el capítulo	5
5.3 — Las promesas.....	5
5.3.1 — Los tres estados, y por qué no se pueden deshacer.....	5
Qué dice	5
En criollo.....	6
5.3.2 — Encadenar: un solo <code>catch</code> para toda la cadena	6
Qué dice	6
En criollo.....	7
5.3.3 — Los cuatro combinadores, y cuándo va cada uno	7
Qué dice	7
En criollo: la distinción que de verdad se usa	7
5.4 — <code>async</code> y <code>await</code>	8
5.4.1 — Qué son exactamente	8
Qué dice	8
En criollo.....	8
5.4.2 — El <code>await</code> adentro de un bucle.....	8
5.5 — <code>fetch</code>	9
5.5.1 — La petición, campo por campo	9
5.5.2 — La respuesta	9
5.5.3 — La decisión más discutida de toda la interfaz.....	9
5.5.4 — Cancelar, y el plazo de espera que <code>fetch</code> no tiene	11
5.6 — La política de mismo origen y CORS	12
5.6.1 — A quién protege, que no es a quien parece	12
Qué dice	12
En criollo.....	12
5.6.2 — La verificación previa	12

5.7 — Los errores: tres categorías distintas.....	13
5.7.1 — La clasificación.....	13
Qué dice	13
En criollo.....	14
5.7.2 — El catálogo del TPI, y dónde se traduce	14
5.7.3 — El reintento y el 429	15
5.8 — El token: dónde vive y qué implica.....	15
La concesión que el TPI hace, y por qué la hace	16
Y RN-F04, que acá se completa	16
5.9 — Los eventos que manda el servidor.....	16
5.9.1 — Por qué SSE y no WebSockets	16
Qué dice	16
En criollo: primero, el abanico completo	16
5.9.2 — El protocolo y su uso.....	17
5.9.3 — El hueco de la reconexión	17
5.9.4 — Las tres reglas.....	18
5.10 — Herramientas de diagnóstico.....	19
5.11 — Seguridad y evolución.....	21
5.12 — Verificación: el checklist honesto.....	21
5.13 — Los doce errores frecuentes	22
5.14 — Las actividades, y qué busca cada una	23
1. Predicción de orden	23
2. Capa <code>api/</code> mínima.....	23
3. Serie contra paralelo	23
4. Las tres categorías	23
5. Cancelación en una búsqueda	23
6. Exploración: el hueco de la reconexión	23
7. Exploración: qué protege CORS.....	23
5.15 — Síntesis: las once frases	24
5.16 — Qué leer, y en qué orden	24
Si lees una sola cosa.....	24
Si lees tres	24
Las fuentes normativas (para consultar, no para leer de corrido)	24
Y del TPI, cuatro secciones para tener abiertas	25
Cierre: las siete cosas que hay que recordar	25

Asincronía y red

Promesas, `fetch`, errores y eventos del servidor, explicados en criollo

Traducción didáctica del texto académico, sección por sección. Mismos conceptos, otro idioma.

Antes de empezar: cómo usar esta guía

Este documento no reemplaza al capítulo original: lo traduce. El texto académico está escrito en el idioma de los papers —denso, comprimido—. Esta guía lo desarma y lo cuenta como se lo contarías a alguien en un café.

La regla es una sola: **no se pierde ni un concepto**. Si el original nombra la RN-F09, acá se nombra la RN-F09. Cada sección conceptual tiene tres partes:

- **Qué dice** — la idea del original, en dos o tres oraciones.
- **En criollo** — la explicación larga, con la analogía que la hace pegar.
- **Para el pizarrón** — la frase que te tenés que llevar.

LA IDEA MADRE DE TODO EL CAPÍTULO

Hay un solo hilo, y ese hilo además dibuja la pantalla. Toda la maquinaria de este capítulo existe para que ese hilo nunca se quede esperando.

Una petición tarda entre decenas y miles de milisegundos, y si el hilo se quedara parado esperándola la página estaría muerta. Las promesas, `async/await` y el bucle de eventos del Capítulo 3 no son tres temas: son tres capas de ese problema.

Y de ahí la advertencia que atraviesa el capítulo: **asincronía no es concurrencia**. No hay dos cosas a la vez; hay una que cede el control y vuelve.

5.1 — De qué se trata esta clase

Qué dice

El Capítulo 1 describió el protocolo que trae los documentos; este estudia cómo se emiten peticiones **desde el código**, sin recargar la página, y cómo se reacciona a lo que el servidor manda por iniciativa propia. El punto de partida es la restricción del Capítulo 3: un solo hilo, que además dibuja la pantalla.

En criollo

En el Capítulo 1 el que emitía la petición era **el navegador**; acá sos vos. Y en la segunda mitad aparece lo que aquel capítulo dejó sin resolver: **qué se hace cuando el que tiene algo para decir es el servidor**.

Si no sabés esto...	...no vas a entender esto otro
Que <code>await</code> no espera : cede el control y vuelve	Por qué una <code>async</code> con un cálculo pesado adentro bloquea igual , y por qué el orden de ejecución no es el de escritura
Que <code>fetch</code> no considera un error un 500	Por qué tu <code>try/catch</code> no atrapa nada y el objeto de error termina procesado como si fuera la lista de productos
Que el mecanismo de eventos del backend no guarda lo que publicó	Por qué existen RN-F09, RN-F10 y RN-F11, y por qué el panel de cocina puede mostrar pedidos viejos sin un mensaje de error

Tres de las once reglas obligatorias del TPI nacen acá. **RN-F09**: un evento nunca escribe datos en la caché de consultas, invalida la clave y deja que se recargue. **RN-F10**: una sola conexión de eventos por sesión. **RN-F11**: la interfaz nunca depende de haber recibido un evento, y declara un intervalo de recarga de respaldo.

Parecen arbitrarias hasta que entendés una limitación que **el propio TPI declara** en su sección 11.3: el mecanismo de publicación del backend **no persiste los mensajes**. Lo que se publica mientras un cliente no está conectado, ese cliente no lo recibe. La sección 5.9.4 hace el razonamiento completo.

PARA EL PIZARRÓN

El objetivo se mide con una función: al terminar tenés que poder escribir una que consulte la API y haga **cuatro cosas a la vez**. Que **distinga un fallo de red de un error de aplicación**, que **se cancele sola** si la vista que la pidió se desmontó, que **traduzca los códigos del catálogo de errores del TPI** —su sección 14.1— a mensajes útiles, y que **lleve plazo de espera**.

Si hace las cuatro, entendiste el capítulo. Si hace tres, te falta la que más caro sale.

5.2 — Por qué existe la petición en segundo plano

Qué dice

Hasta fines de los noventa, toda interacción con un servidor implicaba **reemplazar la página entera**. La técnica que lo cambió nació en Microsoft en 1999, existió seis años sin nombre y se volvió ubicua cuando un ensayo de 2005 la bautizó. De ahí salen cuatro decisiones de diseño que gobiernan el capítulo.

En criollo: la génesis, en tres movimientos

Antes. Marcar una casilla o agregar un producto al carrito hacía que **la página entera se fuera y viniera otra**: parpadeo en blanco, posición de desplazamiento perdida y lo escrito en otros campos descartado. Y cada interacción **transmitía el documento completo** —cabecera, menú, pie— aunque hubiera cambiado una línea.

El invento. Microsoft necesitaba que **Outlook Web Access** se pareciera al cliente de escritorio, y en 1999 le agregó a Internet Explorer 5 un objeto que emitía una petición HTTP desde el código: **XMLHTTP**, como componente ActiveX. Mozilla lo implementó de forma nativa con el nombre **XMLHttpRequest** y los demás lo copiaron. Fijate en el orden, que es el de casi toda la plataforma: **se estandarizó años después de ser universal**, y el nombre quedó pegado a XML aunque casi nunca se usó para eso.

El nombre. En febrero de 2005 Jesse James Garrett publicó *Ajax: A New Approach to Web Applications*, que la bautizó y mostró que Google Maps y Gmail ya la usaban a gran escala. **La técnica tenía seis años; el nombre le dio visibilidad.**

El problema siguiente fue de forma, no de capacidad

XMLHttpRequest se programaba con funciones de retorno, y como cada petición dependía del resultado de la anterior, el código se anidaba:

```
obtenerUsuario(id, function (usuario) {
  obtenerDirecciones(usuario.id, function (direcciones) {
    obtenerPedidos(usuario.id, function (pedidos) {
      // tres niveles, y falta manejar los errores de cada uno
    }, alFallar);
  }, alFallar);
}, alFallar);
```

```
}, alFallar);
```

Se lo llamó **infierno de funciones de retorno**, y su problema real **no era la indentación**: era el manejo de errores. **Cada nivel necesitaba su propio tratamiento** —contá los `alFallar`—, **no había forma de capturar un fallo de todo el conjunto**, y una excepción lanzada adentro de una función de retorno **no se podía atrapar desde afuera**.

Las **promesas** lo resolvieron con una idea de una línea: **representar el resultado futuro como un valor**. Un valor se guarda, se pasa, se devuelve y se combina; una función de retorno no, porque es una instrucción que le dejás a otro. Es el **ticket de la tintorería**: no te llevás el saco, te llevás un papel, y con el papel podés hacer cosas que con el saco todavía no.

Se especificaron como acuerdo comunitario —Promises/A+, 2012— y entraron al lenguaje en ES2015. `async/await` llegó en ES2017 para escribirlas con la forma del código secuencial, y `fetch` reemplazó a `XMLHttpRequest` con una interfaz basada en promesas.

Las cuatro decisiones que explican todo el capítulo

Decisión	Qué gana	Qué cuesta
1. La asincronía no es concurrencia: la operación cede el control y se reanuda después, por el bucle de eventos de la 3.10	Un modelo simple, sin carreras entre hilos	Que el orden de ejecución no es el de escritura
2. El resultado futuro es un valor	Se pueden componer operaciones: los combinadores de la 5.3.3	Que una promesa no se puede reiniciar ni cancelar (5.3.1)
3. <code>fetch</code> no considera un error el del servidor: un 404 o un 500 cumplen la promesa	Coherencia: modela el transporte, y ahí todo salió bien	La fuente de errores más frecuente del capítulo (5.5.3)
4. El navegador restringe por origen de forma determinada	Que un sitio cualquiera no lea tu correo con tu sesión	Verificaciones previas y errores que no se arreglan desde el frontend (5.6)

PARA ENTENDER: ``await`` no significa «esperá acá»

Significa: «*guardá dónde estoy, soltá el hilo para que haga otra cosa, y volvé a este punto cuando llegue el resultado*».

Pensalo como un restaurante con **un solo mozo**: lleva tu pedido a la cocina y **no se queda parado en la ventanilla**, atiende otras mesas y vuelve cuando el plato está listo. No hay dos mozos: hay uno que se va y vuelve.

Y de ahí algo práctico: si adentro de una `async` hacés un cálculo pesado y sincrónico, **la función es `async` y bloquea igual**. Sólo el `await` cede el control, y sólo donde está escrito.

5.3 — Las promesas

5.3.1 — Los tres estados, y por qué no se pueden deshacer

Qué dice

Una promesa está siempre en uno de tres estados; la transición es de una sola vía y las continuaciones se ejecutan como microtarear.

Estado	Significado
Pendiente	La operación no terminó

Estado	Significado
Cumplida	Terminó bien y tiene un valor
Rechazada	Terminó mal y tiene un motivo

En criollo

De una sola vía: una vez resuelta, su estado y su valor **no cambian nunca más**. Es un partido ya jugado: se puede discutir, no cambiar.

Como microtarear: por el algoritmo de la 3.10.2, **una promesa ya resuelta siempre se procesa antes que un `setTimeout(0)`**, porque las microtarear se vacían enteras antes de la próxima tarea. Por eso el orden de tu consola te sorprende.

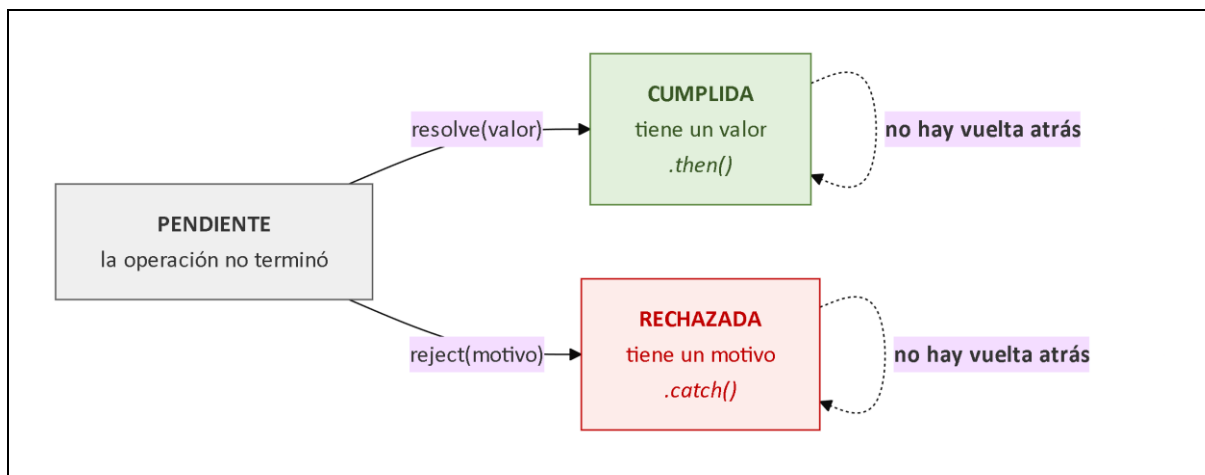


Figura 5.1. Estados de una promesa

La inmutabilidad tiene un precio: **una promesa no se puede reiniciar ni cancelar**. No cancelás un ticket de tintorería; llamás a la tintorería. Lo que se cancela es **la operación subyacente**, con el mecanismo de la 5.5.4.

Y llamar a la función que crea la promesa dispara la operación de inmediato, aunque nadie haga `await`.

```
const promesa = obtenerPedido(id); // la petición YA se emitió acá
// ... otro código ...
const pedido = await promesa;      // acá sólo se espera el resultado
```

Guardar una promesa **no es guardar una intención de pedir algo**: el pedido ya salió. Y eso es la herramienta que permite **lanzar varias operaciones y esperarlas después** — la base de los combinadores de la 5.3.3.

5.3.2 — Encadenar: un solo `catch` para toda la cadena

Qué dice

`then` devuelve **una promesa nueva**, y de ahí sale el encadenamiento. Un solo manejador de error cubre la cadena completa.

```
obtenerUsuario(id)
  .then(usuario => obtenerPedidos(usuario.id)) // devuelve otra promesa
  .then(pedidos => mostrar(pedidos))
  .catch(error => manejar(error))             // captura cualquiera de los tres
  .finally(() => ocultarCargando());           // se ejecuta siempre
```

En criollo

Mirá la línea del `catch` y compará con los tres `catch` de la 5.2: **ésa es toda la diferencia**. Si cualquiera de los pasos falla, el control salta ahí, igual que una excepción sube por la pila en código sincrónico.

`finally` se ejecuta **termine bien o mal**: es el lugar correcto para ocultar un indicador de carga, porque si lo ocultás sólo en el camino feliz, el día que falle el usuario se queda mirando un spinner eterno.

5.3.3 — Los cuatro combinadores, y cuándo va cada uno

Qué dice

Cuatro funciones combinan varias promesas, y elegir la equivocada produce comportamientos difíciles de diagnosticar.

Combinador	Espera a	Falla si	Devuelve	
<code>Promise.all</code>	Todas	Falla una	Arreglo de valores	
<code>Promise.allSettled</code>	Todas	Nunca	Arreglo de <code>{status, value \ reason}</code>	
<code>Promise.race</code>	La primera en resolverse	Si esa primera falla	El valor de esa	
<code>Promise.any</code>	La primera cumplida	Fallan todas	El valor de esa	

En criollo: la distinción que de verdad se usa

`Promise.all` falla apenas una falla y descarta los resultados de las que sí funcionaron: es una receta, si falta la harina no hay torta. Sirve cuando el conjunto **no tiene sentido incompleto**: no hay pantalla de pedido sin el pedido.

`Promise.allSettled` espera a todas y devuelve el resultado de cada una, exitoso o no: es el tablero de llegadas del aeropuerto, donde un vuelo cancelado no borra la información de los otros siete.

```
const [productos, categorias] = await Promise.all([
  listarProductos(), listarCategorias()
]); // sin categorías no hay catálogo que mostrar

const resultados = await Promise.allSettled([
  ventasDelDia(), pedidosPendientes(), stockCritico(), ticketPromedio()
]); // cada tarjeta se muestra o falla por su cuenta
```

`Promise.race` devuelve la primera que se resuelva, gane o pierda —se usaba para armar plazos de espera a mano, y hoy hay algo mejor en la 5.5.4—; **`Promise.any` devuelve la primera que se cumpla**, ignorando las que fallen.

OJO ACÁ: el error caro no es elegir mal el combinador

Es **no usar ninguno**.

```
const productos = await listarProductos();
const categorias = await listarCategorias();
```

Se ve prolijo y es **el doble de lento de lo necesario**: la segunda no arranca hasta que termina la primera, aunque no dependa de ella. Con 300 ms cada una tenés 600 ms de espera para algo que podía tardar 300.

Regla: si dos peticiones no dependen entre sí, van juntas en un combinador. Y ojo, que **este error un agente de IA te lo va a escribir siempre**: es lo que sale de traducir «traé A y traé B».

5.4 — `async` y `await`

5.4.1 — Qué son exactamente

Qué dice

Una función `async` **siempre devuelve una promesa**, aunque su cuerpo devuelva un valor común. `await` suspende la función hasta que la promesa se resuelva y devuelve su valor; si se rechaza, **lanza** el motivo como excepción, lo que permite usar `try/catch`.

```
async function cargarCatalogo() {
  try {
    const productos = await listarProductos();
    mostrar(productos);
  } catch (error) {
    mostrarError(error);
  } finally {
    ocultarCargando();
  }
}
```

En criollo

Lo valioso de la sintaxis es que `await` convierta el rechazo en excepción. Antes, un error asincrónico **no se podía capturar con `try/catch`**: para cuando ocurría, el bloque `try` ya había terminado. Con `await`, la función **se reanuda lanzando** en el punto donde te fuiste. `async/await` **no agrega capacidad, agrega legibilidad**: debajo son las mismas promesas de la 5.3.

5.4.2 — El `await` adentro de un bucle

Es **el error de rendimiento más común del código asincrónico**, y no hay nada que un linter pueda marcarte.

```
// Serializa: cada petición espera a la anterior
for (const id of idsDePedidos) {
  const pedido = await obtenerPedido(id);
  procesar(pedido);
}

// Paraleliza: todas salen juntas
const pedidos = await Promise.all(idsDePedidos.map(id => obtenerPedido(id)));
pedidos.forEach(procesar);
```

Con **veinte pedidos y 200 milisegundos cada uno**, la primera versión tarda **cuatro segundos** y la segunda **doscientos milisegundos**. Sintácticamente está bien: está mal porque **serializa algo que no necesitaba ser serializado**. Es la cola del banco con veinte cajas abiertas y todo el mundo formado en una sola.

La versión secuencial corresponde en dos casos: **cuando cada petición depende de la anterior**, y **cuando veinte peticiones simultáneas superarían un límite del servidor**, como el de la 5.7.3 — ahí no dan veinte respuestas: dan un 429.

NOTA: `forEach` no espera nada

```
pedidos.forEach(async (p) => {
```

```

    await guardar(p);
  });
  console.log("listo");    // se imprime ANTES de que se guarde ninguno
  // forEach no sabe qué hacer con la promesa que le devuelve la función: la ignora y sigue, con las
  // veinte operaciones en el aire. Lo grave no es el console.log: es cuando lo que sigue es «mostrar
  // el mensaje de éxito».

  for (const p of pedidos) await guardar(p);           // de a uno, en orden
  await Promise.all(pedidos.map(p => guardar(p)));      // todas juntas
  // for...of sí respeta el await; forEach, map y compañía, no: el primero es sintaxis del lenguaje y
  // el motor sabe suspenderla, el segundo es una función común que tira a la basura lo que le
  // devuelven.

```

5.5 — fetch

5.5.1 — La petición, campo por campo

```

const respuesta = await fetch("/api/v1/pedidos", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "Authorization": `Bearer ${token}`,
    "Idempotency-Key": crypto.randomUUID(),
  },
  body: JSON.stringify({ direccion_id: 42, items }),
});

```

Esto es la **anatomía de la sección 1.5 escrita en JavaScript**. `Authorization` con esquema `Bearer` es la pulsera de la 1.4.1: el token que viaja **en cada petición**, porque el protocolo no se acuerda de nada. El `Idempotency-Key` es el de la 1.4.2, el que **RN-F07** exige en el checkout — y `crypto.randomUUID()` es una función **del navegador**, no de una biblioteca.

En el `body` está el detalle que más errores produce: **el cuerpo tiene que ser una cadena**. Si le pasás un objeto, `fetch` no falla con un mensaje claro: manda el resultado de convertirlo a texto, que es `[object Object]`. El servidor contesta un 400 que vas a buscar media hora en el cliente, donde no está.

5.5.2 — La respuesta

```

respuesta.ok           // true si el código está entre 200 y 299
respuesta.status       // el número: 200, 404, 500
respuesta.headers      // los encabezados
await respuesta.json(); // parsea el cuerpo como JSON

```

`respuesta.ok` es «el código está en la familia 2xx» de la 1.4.3. Y algo que sorprende: el cuerpo llega **como flujo y se puede consumir una sola vez**; un segundo intento lanza un error. Un flujo es agua que pasa, no un archivo que queda. Si necesitás leerlo dos veces —intentar `json()` y caer a `text()`—, hay que **clonar la respuesta antes** con `respuesta.clone()`.

5.5.3 — La decisión más discutida de toda la interfaz

Acá está la fuente de errores más frecuente de todo el capítulo:

fetch no rechaza la promesa ante un error HTTP. Un 404, un 422 o un 500 **cumplen** la promesa.

```

// MAL: el catch nunca se ejecuta ante un 500
try {

```

```
const r = await fetch("/api/v1/pedidos/9999");
const pedido = await r.json(); // el cuerpo es el error, no un pedido
} catch (e) {
  // sólo llega acá si falló la RED
}
```

`fetch` rechaza **únicamente en tres situaciones**, las tres únicas veces que tu `catch` se va a ejecutar: **fallo de red** —no hubo respuesta—, **bloqueo por origen** (sección 5.6) y **cancelación** (sección 5.5.4).

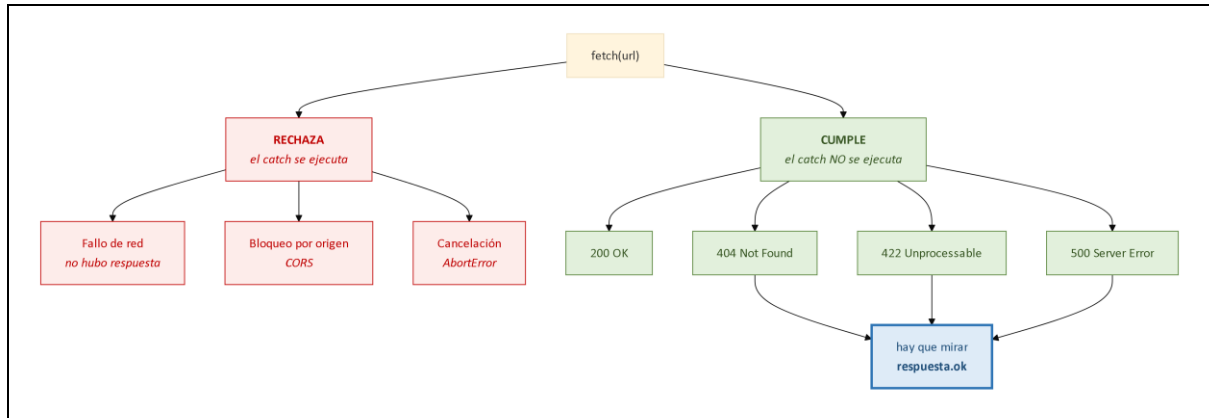


Figura 5.2. Cuándo `fetch` rechaza y cuándo cumple

La lógica existe aunque moleste: **un 404 es una respuesta exitosa del protocolo**. Es el correo: **el cartero entregó la carta e hizo su trabajo perfecto**, y que la carta diga «su solicitud fue rechazada» no es problema del correo. Que la respuesta comunique un problema de la aplicación es información, no un fallo de la operación de red.

Coherente o no, obliga a comprobar siempre:

```
const respuesta = await fetch(url, opciones);
if (!respuesta.ok) {
  const cuerpo = await respuesta.json().catch(() => null);
  throw new ErrorDeApi(respuesta.status, cuerpo?.code, cuerpo?.message);
}
return respuesta.json();
```

Lo importante no es el código sino **dónde va: se escribe una sola vez**, en la capa `api/` que el Capítulo 8 estudia. Repetirla en cada vista garantiza que en algún lado va a faltar — justo en la que nadie probó.

NOTA: el TPI declara `Axios`, no `fetch`

¿Y entonces por qué estudiamos `fetch`? Porque es **la interfaz de la plataforma** —Axios está construido sobre ella— y porque el comportamiento que acabás de ver es el que Axios decidió cambiar: **Axios sí rechaza ante un 4xx o un 5xx**, y el error trae `error.response.status` y `error.response.data`.

No son dos interfaces distintas: son dos decisiones distintas sobre el mismo problema. Lo que **no** cambia es el principio de la 5.7: distinguir red, protocolo y aplicación sigue siendo tu trabajo. Y el TPI usa sus **interceptores**, donde el token y la traducción de errores se aplican una sola vez para toda la aplicación.

OJO ACÁ: el error más común del código que escriben los agentes

Cuando le pidas «escribime una función que traiga los productos», va a salir esto:

```
try {
  const r = await fetch("/api/v1/productos");
  return await r.json();
}
```

```

    } catch (e) {
      mostrarError("No se pudieron cargar los productos");
    }

```

Se ve perfecto. Tiene su try/catch. **Y no maneja ningún error del servidor.**

Si el backend responde 401 porque venció el token, ese catch no se ejecuta: la función devuelve el objeto de error parseado como si fuera la lista de productos, y el bug aparece tres capas más arriba. Vas a depurar el componente de la lista, que es el único lugar donde el problema **no** está.

Si no ves un if (!respuesta.ok), el código está mal.

5.5.4 — Cancelar, y el plazo de espera que fetch no tiene

El AbortController de la sección 4.9.4 —el mismo objeto con el que dabas de baja escuchas de eventos— **también cancela peticiones**:

```

const controlador = new AbortController();
fetch(url, { signal: controlador.signal });

controlador.abort(); // la promesa se rechaza con un AbortError

```

El caso que lo hace necesario: **una vista que se desmonta mientras espera una respuesta**. La respuesta llega igual e intenta actualizar un elemento que **ya no está en el documento**. En el mejor caso falla en silencio; en el peor, la continuación retiene los nodos de la vista vieja y **todo el subárbol sigue vivo en memoria**. Eso es la fuga de la sección 4.9: **un pedido de delivery a una casa de la que ya te mudaste**.

PARA ENTENDER: cancelar no es ahorrar — es RN-F01 otra vez

RN-F01 exige que toda suscripción guarde su función de baja y la ejecute al destruir el componente. La escribiste pensando en addEventListener, pero **una petición en vuelo es exactamente lo mismo**. Y el mecanismo es uno solo:

```

const controlador = new AbortController();
elemento.addEventListener("click", alHacerClic, { signal: controlador.signal });
fetch(url, { signal: controlador.signal });

controlador.abort(); // se van las dos cosas de una

```

Un controlador por vista y abort() al desmontar cubre las escuchas y las peticiones; por eso el Capítulo 7 mete las dos en la misma clase base. **La cancelación no es una optimización: es corrección.**

El otro caso es la **búsqueda mientras se escribe**: cada tecla dispara una petición y las respuestas pueden llegar **desordenadas**, de modo que un resultado viejo pise a uno nuevo.

Para el plazo de espera hay una función nativa que evita armarlo a mano:

```

fetch(url, { signal: AbortSignal.timeout(5000) });

```

Hace falta porque **fetch no tiene plazo de espera propio**. Sin uno, una petición queda pendiente hasta que el sistema operativo la corte, que pueden ser minutos.

OJO ACÁ: la peor falla es la que no falla

El usuario aprieta «Confirmar pedido» con mala señal. La petición sale y **queda colgada**: el botón sigue en «Enviando...», no hay error que mostrar porque no hubo error, y el catch no se ejecuta porque nada se rechazó. El usuario espera treinta segundos, se cansa, y **aprieta de nuevo**.

Y ahí tenés dos pedidos. Por eso RN-F07 —la clave de idempotencia del Capítulo 1— existe: porque este escenario **va a pasar**. El plazo de espera evita que el usuario quede colgado; la clave lo protege el día que igual apriete dos veces.

Regla: **toda petición lleva plazo de espera**. Un error explícito a los cinco segundos es infinitamente mejor que un botón que gira para siempre.

5.6 — La política de mismo origen y CORS

5.6.1 — A quién protege, que no es a quien parece

Qué dice

Dos direcciones tienen el **mismo origen** si coinciden en **esquema, host y puerto**. Por defecto, un documento no puede leer respuestas de otro origen. La política no protege al servidor: protege al usuario.

En criollo

Los tres tienen que coincidir: `https://foodstore.example` y `http://foodstore.example` son orígenes distintos, y también lo son el puerto 5173 y el 8000 — por eso en desarrollo te comés un error de CORS el primer día.

La razón se ve con un escenario. Alguien tiene la sesión abierta en su banco y en otra pestaña visita un sitio cualquiera, que desde el navegador de la víctima emite una petición al banco: **viaja con las cookies de la víctima** y el banco contesta el saldo. **Sin la política de mismo origen, eso funcionaría.**

De ahí la precisión que casi nadie tiene clara: **CORS no protege al servidor. Protege al usuario de que un sitio lea respuestas de otro sitio usando sus credenciales**. El servidor debe autorizar cada petición por su cuenta —con el token del Capítulo 1—, porque un programa fuera del navegador ignora CORS: **un `curl` no tiene política de mismo origen**.

5.6.2 — La verificación previa

Algunas peticiones se emiten directamente y otras van precedidas de una consulta. La distinción es **histórica**: las que un formulario HTML podía hacer **antes de que CORS existiera** se admiten sin consulta, porque prohibirlas habría roto la web.

Se emite directo	Requiere verificación previa
GET, HEAD, POST	Cualquier otro método: PUT, PATCH, DELETE
Sin encabezados propios	Con encabezados propios: Authorization, Idempotency-Key
Content-Type de formulario o texto plano	Content-Type: application/json

Mirá la columna derecha y después el código de la 5.5.1: **casi toda petición a la API del TPI dispara una verificación previa**, porque manda JSON y lleva `Authorization`.

Esa verificación es una petición `OPTIONS` que **el navegador emite solo**, declarando qué método y qué encabezados pretende usar; el servidor responde qué admite y **sólo entonces** se emite la petición real. El encabezado `Access-Control-Max-Age` le permite al navegador recordar la respuesta y no repetir la consulta cada vez.

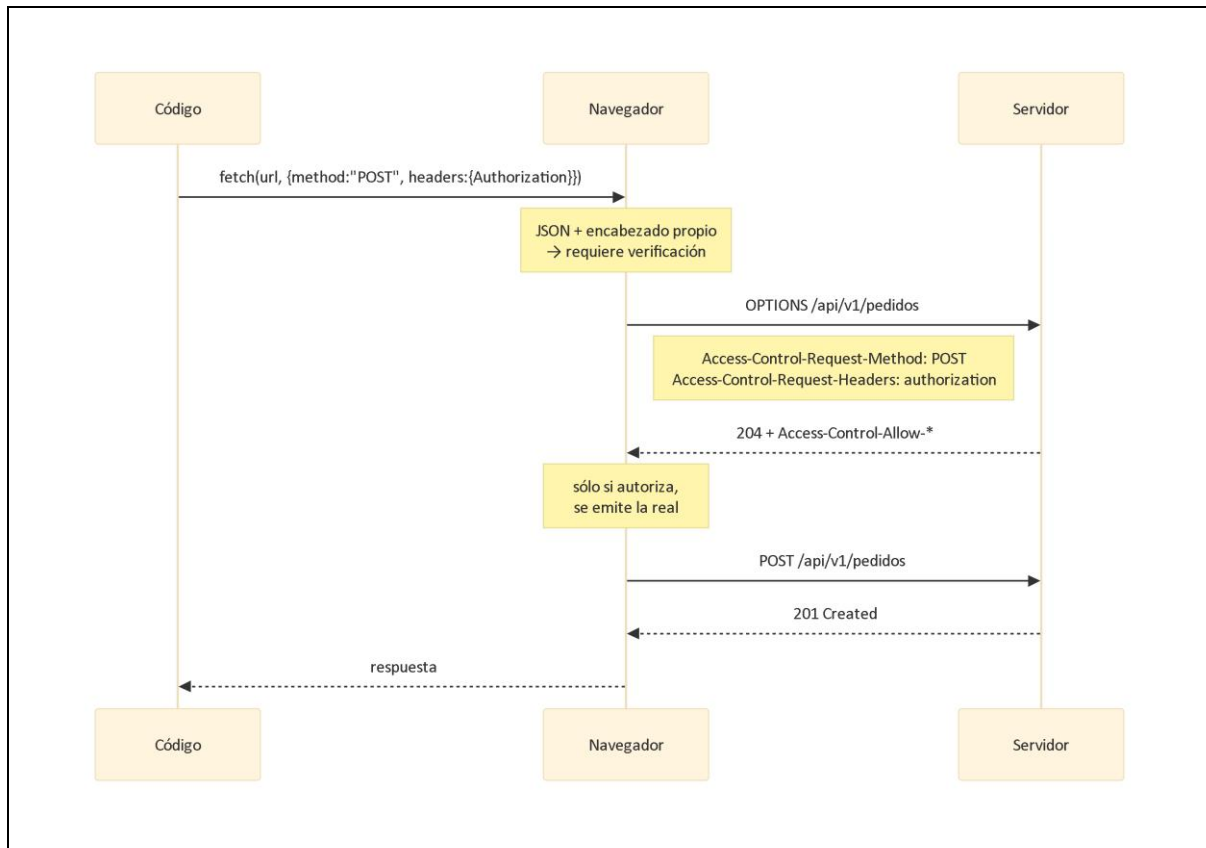


Figura 5.3. La verificación previa de CORS

Eso explica un detalle que confunde al diagnosticar: **en el panel de red aparecen dos peticiones donde tu código escribió una sola**. La primera, `OPTIONS`, la emitió el navegador. Y si esa primera falla, la segunda **nunca se emite**, de modo que el servidor **jamás se entera** de que alguien quiso hacer algo.

Cuando se envían cookies la configuración se vuelve estricta: **el servidor no puede responder con el comodín ***; debe devolver el origen exacto y declarar explícitamente que admite credenciales.

PARA ENTENDER: tres cosas sobre los errores de CORS

Una: el error es del navegador, no del servidor. El servidor contestó perfecto; el navegador recibió la respuesta, no encontró la autorización y **decidió no dejarte leerla**. Por eso el mensaje aparece en la consola y no en el panel de red. Es el mensajero que trajo el sobre y no te lo entrega porque el remitente no puso tu nombre en la lista.

Dos: la petición sí llegó. En una petición simple el servidor la procesó: si era un `POST` que creaba algo, **lo creó**.

Tres, y es la que importa: CORS se arregla en el servidor. Siempre. Cuando encuentres un plugin que «desactiva CORS», entendé lo que hace: **lo desactiva para vos, en tu máquina**. Tus usuarios lo van a seguir teniendo.

5.7 — Los errores: tres categorías distintas

5.7.1 — La clasificación

Qué dice

Un cliente serio distingue tres cosas que suelen tratarse igual.

Categoría	Qué pasó	Cómo se detecta	¿Reintentar?
Red	La petición no llegó o no volvió	<code>fetch rechaza</code>	Sí, con espera creciente
Protocolo	Llegó y el servidor falló	<code>!respuesta.ok</code> con <code>5xx</code>	A veces
Aplicación	Llegó, se entendió, y la operación no procede	<code>!respuesta.ok</code> con <code>4xx</code>	No

En criollo

La tabla continúa la 1.4.3 —4xx sos vos, 5xx soy yo— y le agrega la fila de arriba, la red, que en el Capítulo 1 no existía porque ahí el que emitía la petición era el navegador.

La tercera fila es la que más se maltrata. Un 409 porque el producto se quedó sin stock **no es un error técnico**: es información de negocio que el usuario necesita ver. Mostrar «error de conexión» ante un 409 **es mentirle**: la conexión anduvo perfecto, y el usuario va a revisar su wifi y reintentar para nada.

5.7.2 — El catálogo del TPI, y dónde se traduce

La sección 14.1 del TPI define un **catálogo de códigos de error**, cada uno con su código HTTP y la situación que lo produce, y eso tiene una consecuencia de diseño importante: **el cliente puede distinguir casos por un código estable en lugar de leer el texto del mensaje**. La sección 14.2 contesta la otra mitad —**dónde** ocurre esa traducción—, y la respuesta es la que este capítulo viene sosteniendo: **en un solo lugar**. Un error que llega a una vista ya debe ser un objeto con significado, no una respuesta HTTP cruda.

```
export class ErrorDeApi extends Error {
  constructor(estado, codigo, mensaje, reintentarEn) {
    super(mensaje);
    this.estado = estado;           // 409
    this.codigo = codigo;           // "STOCK_INSUFICIENTE"
    this.reintentarEn = reintentarEn; // de Retry-After, si vino
  }
  get esDeNegocio() { return this.estado >= 400 && this.estado < 500; }
  get esReintentable() { return this.estado >= 500 || this.estado === 429; }
}
```

Con eso, la vista deja de mirar números y pasa a **preguntar por significado**:

```
try {
  await crearPedido(datos);
} catch (error) {
  if (error.codigo === "STOCK_INSUFICIENTE") mostrarFaltanteDeStock(error);
  else if (error.esDeNegocio) mostrarAviso(error.message);
  else mostrarErrorGenerico();
}
```

Mirá lo que esa vista **no** hace: no revisa `respuesta.ok`, no parsea el cuerpo, no compara textos. Todo eso ocurrió **una sola vez**, en la capa `api/`.

El código es estable; el mensaje, no

"No hay stock suficiente" se puede corregir por una falta de ortografía, traducir o reescribir para que suene más amable: las tres son cosas **legítimas** que el backend hace sin avisar.

Y sin embargo, si tu vista dice `if (error.message === "No hay stock suficiente")`, cada uno de esos cambios **rompe la aplicación en silencio**: el `if` deja de dar verdadero y no hay error en ningún log.

Es la diferencia entre el código postal y el nombre de la calle. **Compará códigos, mostrá mensajes.**

5.7.3 — El reintento y el 429

Ante un 429, el servidor indica cuánto esperar mediante el encabezado `Retry-After` de la 1.4.3. El TPI lo usa en su límite de intentos de autenticación, descrito en su sección 4.4.

Un cliente que ignora ese encabezado y reintenta de inmediato empeora exactamente el problema que el límite intenta contener. Y como el límite del TPI cuenta **por dirección de red además de por cuenta**, un cliente mal escrito puede dejar sin servicio a **todos los usuarios de la misma red**.

Cuando el reintento corresponde —fallo de red o 5xx—, la práctica correcta es esperar cada vez más entre intentos, con una **pequeña variación aleatoria**. Eso último es lo que más se olvida: sin la variación, todos los clientes que fallaron a la vez reintentan a la vez y el servidor que se estaba recuperando **recibe otra avalancha**. Es el aplauso: si todos aplauden al mismo tiempo, el ruido es máximo.

5.8 — El token: dónde vive y qué implica

El Capítulo 1 estableció que el token viaja en cada petición. Queda por resolver **dónde se guarda entre una y otra**, y **ninguna opción es enteramente satisfactoria**.

Lugar	Sobrevive al cierre	Accesible por script	Se envía solo
localStorage	Sí	Sí	No
sessionStorage	No	Sí	No
Cookie común	Sí	Sí	Sí
Cookie <code>HttpOnly</code>	Sí	No	Sí
Memoria	No	Sí	No

Las tres primeras filas comparten un riesgo: **si hay un XSS, el token se puede leer**, que es exactamente el ataque de la sección 4.7.2. La cookie `HttpOnly` es inaccesible desde el código, pero al enviarse automáticamente introduce **otro** problema —la falsificación de petición entre sitios— que exige su propia defensa.

No hay una opción sin costo, y por eso la defensa de fondo no es dónde se guarda el token sino **que no haya XSS**: RN-F02 otra vez.

OJO ACÁ: si tenés un XSS, ya perdiste

Vas a encontrar mucha discusión sobre dónde guardar el token. Quedate con esto: **si tenés un XSS, ya perdiste, guardes el token donde lo guardes.**

¿Lo tenés en una cookie `HttpOnly` que el script no puede leer? El atacante no lee el token — **hace las peticiones desde tu propia página, con tu sesión**. No necesita robarlo: le alcanza con usarlo. Es el ladrón que ya está adentro de tu casa: **da igual qué tan buena sea la cerradura.**

La discusión cambia **qué** puede hacer el atacante, no **si** puede hacer algo.

La concesión que el TPI hace, y por qué la hace

Su sección 11.4 establece que, como la interfaz nativa de eventos **no permite fijar encabezados** —no hay forma de mandarle un `Authorization` a un `EventSource`—, el token viaja **como parámetro de consulta**, y **únicamente en el endpoint de eventos**.

Eso contradice lo que el Capítulo 1 advirtió sobre no poner datos sensibles en la cadena de consulta, y la contradicción **es real**: es una concesión a una limitación de la plataforma, **acotada a un solo endpoint** y acompañada de mitigaciones del lado del servidor. Guardate la forma de esa decisión: cuando la plataforma no te deja hacer lo correcto, se elige el mal menor, **se acota y se documenta**.

Y RN-F04, que acá se completa

Las guardas de ruta son usabilidad, no seguridad: el backend revalida siempre el rol y la propiedad del recurso.

Ocultar un botón de administración evita que un usuario común se confunda. **No evita nada más**. Cualquiera puede emitir la petición con `curl` —que, como vimos en la 5.6.1, ni siquiera tiene política de mismo origen—, y el único lugar donde esa petición se puede rechazar es el servidor.

5.9 — Los eventos que manda el servidor

5.9.1 — Por qué SSE y no WebSockets

Qué dice

El Capítulo 1 dejó planteado que el servidor no puede iniciar la conversación. El TPI documenta en su sección 11.1 los cinco criterios por los que eligió la alternativa menos conocida.

En criollo: primero, el abanico completo

«El servidor no puede hablar primero» se sorteó históricamente de cuatro maneras, y cada una resigna algo:

Mecanismo	Cómo hace que el servidor «hable»	Qué cuesta
Consulta periódica (<i>polling</i>)	El cliente pregunta cada N segundos si hay novedad	Casi todo es «no», y la novedad llega tarde: medio intervalo
Consulta larga (<i>long polling</i>)	El servidor no contesta hasta que hay novedad	Una conexión ocupada por cliente, y reconectar tras cada respuesta
Eventos del servidor (SSE)	Deja una respuesta abierta y escribe eventos	Canal unidireccional y límite de conexiones por dominio en HTTP/1.1
WebSocket	Cambia de protocolo: canal bidireccional permanente	Ya no es HTTP: infraestructura propia y reconexión a mano

El TPI eligió el tercero, y sus cinco criterios explican por qué:

Criterio	Por qué favorece a SSE
Dirección	El flujo es unidireccional: el servidor avisa y el cliente no manda nada por ese canal , porque todo lo que quiere hacer ya tiene su endpoint REST. Un canal bidireccional resolvería un problema que no existe

Criterio	Por qué favorece a SSE
Protocolo	SSE es HTTP: una respuesta común que no se termina de cerrar. Pasa por los mismos intermediarios, infraestructura y autenticación; WebSocket exige un cambio de protocolo que hay que habilitar aparte
Reconexión	La interfaz nativa reconecta sola y reenvía el identificador del último evento, sin escribir una línea. Con WebSocket eso es código propio
Costo por conexión	Sobre un backend asíncrono, una conexión abierta cuesta una corrutina y un socket : una función suspendida, no un hilo esperando
Lo que se pierde	El cliente no puede mandar nada por el canal, y hay un límite de seis conexiones por dominio en HTTP/1.1 — irrelevante acá porque el frontend abre una sola , que es RN-F10

La analogía conviene dibujarla: **SSE es la radio; WebSocket es el teléfono.**

5.9.2 — El protocolo y su uso

El formato es **texto plano**. Cada evento es un bloque de líneas con etiqueta, y un **doble salto de línea** lo cierra:

```
id: 1043
event: pedido_actualizado
data: {"pedido_id": 1043, "estado": "en_preparacion"}
```

Del lado del cliente, la interfaz nativa es igual de breve:

```
const canal = new EventSource(`/api/v1/eventos?token=${token}`);

canal.addEventListener("pedido_actualizado", (evento) => {
  const datos = JSON.parse(evento.data);    // data siempre es texto
  invalidar(["pedidos", datos.pedido_id]);  // ver RN-F09
});

canal.addEventListener("error", () => { /* reconecta solo */ });
```

Dos detalles: **event.data es siempre una cadena** y hay que parsearla, y el token en la dirección corresponde a la concesión de la sección 5.8.

5.9.3 — El hueco de la reconexión

Acá está la limitación que funda las tres reglas, y el TPI la declara **sin disimulo** en su sección 11.3: **el mecanismo de publicación del backend no persiste los mensajes**. Volvé a la radio y se entiende sola: **lo que se transmitió mientras tenías la radio apagada, no lo escuchaste** — y, peor, no tenés forma de saber que te lo perdiste.

El TPI resuelve el hueco **del lado del servidor**, y el mecanismo define qué puede esperar el cliente:

Momento	Qué hace el servidor	Qué puede esperar el cliente
Conexión inicial	Emite un evento de sincronización	Arrancar con el estado al día

Momento	Qué hace el servidor	Qué puede esperar el cliente
Reconexión	El navegador reenvía el identificador del último evento y el servidor emite lo pendiente desde ahí	Recuperar lo perdido si la desconexión fue corta
Desconexión larga	Pasado cierto límite emite un evento de resincronización	Recargar desde cero: los intermedios no vuelven
Conexión ociosa	Manda un comentario periódico cada quince segundos	Que el canal siga vivo: sin eso un intermediario la cierra y empieza la reconexión permanente

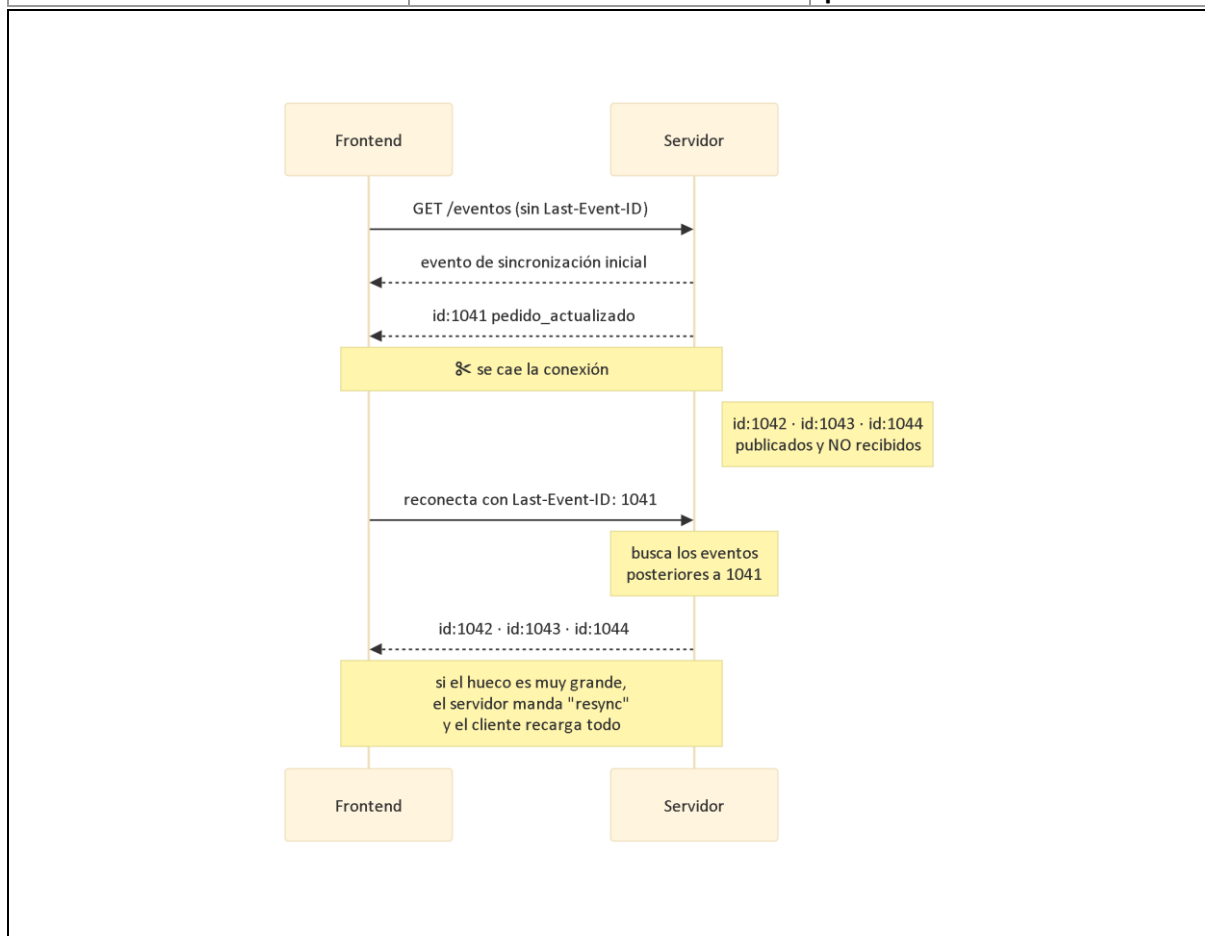


Figura 5.4. El hueco de la reconexión

Ninguno de esos mecanismos garantiza la entrega en todos los casos. El propio diseño admite que puede haber un salto. **De ahí las tres reglas.**

5.9.4 — Las tres reglas

Fijate que recién ahora, después de ver la limitación funcionando, las reglas se pueden enunciar. Antes habrían sido tres caprichos.

RN-F09 — el evento invalida, no escribe.

Un evento recibido por SSE nunca escribe datos en la caché de consultas: invalida la clave correspondiente y deja que se recargue. El evento dice **qué** cambió, no **cuál** es el valor nuevo.

Si un evento se pierde y escribís la caché con los que **sí** llegaron, la interfaz muestra un estado **inventado**, que no corresponde a ningún momento real del servidor; invalidar y recargar siempre produce uno consistente. La analogía: **el evento es el timbre, no el paquete**.

```
// MAL: si se perdió un evento intermedio, el estado queda inventado
canal.addEventListener("pedido_actualizado", (e) => {
  const d = JSON.parse(e.data);
  cache.set(["pedidos", d.pedido_id], d);
});

// BIEN: el evento sólo dice qué mirar de nuevo
canal.addEventListener("pedido_actualizado", (e) => {
  const d = JSON.parse(e.data);
  cache.invalidar(["pedidos", d.pedido_id]);
});
```

RN-F10 — una sola conexión por sesión.

La conexión de eventos es una sola por sesión: se abre en el arranque si hay sesión, contra el único endpoint de la sección 6.11, y se cierra en el logout.

Dos razones. **El límite de seis conexiones por dominio sobre HTTP/1.1**: si cada vista abre la suya, el efecto no es que falle el canal — es que **el resto de la aplicación se queda sin poder hacer peticiones**. Y **el diseño del servidor**: su sección 11.4 establece que el servidor resuelve a qué canales suscribir a cada actor y **multiplexa todo por la misma conexión**.

RN-F11 — la interfaz nunca depende del evento.

Toda vista que muestra datos vivos declara además su intervalo de recarga de respaldo, más largo; si el canal se cae, la pantalla se actualiza igual.

Si el canal puede fallar y el mecanismo no garantiza la entrega, **una interfaz que sólo se actualiza por eventos puede quedarse mostrando información vieja indefinidamente**, sin que nadie lo note. De ahí la frase que resume la sección: **el canal es una optimización de latencia, no la fuente de verdad**. Con canal la pantalla se actualiza en el momento; sin canal, más lento; **nunca deja de actualizarse**. Es el reloj de la estación que se paró: no muestra un error, **muestra una hora**, y vos le creés hasta que perdés el tren.

PARA ENTENDER: cómo una limitación se convierte en tres reglas

El mecanismo de publicación no persiste → un evento se puede perder → por lo tanto: no escribas con lo que llega (**RN-F09**), porque construirías un estado que nunca existió; no abras más canales de los necesarios (**RN-F10**), porque cada uno es otra cosa que se puede caer; no dependas del canal (**RN-F11**), porque se va a caer alguna vez.

Y lo importante para el TPI: **el panel de cocina va a funcionar perfecto en tu demo de quince minutos aunque violes las tres**. Se rompe a las tres horas de turno, cuando el wifi de la cocina parpadea y la pantalla queda mostrando pedidos viejos. **Y nadie se va a dar cuenta**, porque no hay ningún error: hay una pantalla que dejó de actualizarse.

5.10 — Herramientas de diagnóstico

El **panel de red** es el instrumento central, igual que en el Capítulo 1 — pero acá se usa distinto: allá mirabas qué pedía el navegador, acá qué pide **tu código**.

Qué mirar	Qué te dice	Cuándo lo usás
El filtro Fetch/XHR	Aísla las peticiones de tu código de las del documento	Siempre que depures la capa <code>api/</code>

Qué mirar	Qué te dice	Cuándo lo usás
La columna de tiempos , desplegada	Separa resolución, conexión, tiempo hasta el primer byte y descarga	Primer byte tardío = el servidor; descarga larga = la red
La limitación de red	Simula una conexión lenta	Única forma práctica de probar los estados de carga y los plazos de espera (5.5.4)
El modo sin conexión	Provoca el fallo de red de la 5.7.1	Para verificar que el cliente lo distingue de un error del servidor

FIGURA 5.5

captura

Captura del panel con la limitación de red activada y la columna de tiempos desplegada, mostrando la separación entre espera del servidor y descarga

Figura 5.5. El panel de red con limitación

Para SSE el panel tiene una vista específica: al seleccionar la conexión del canal aparece una pestaña de **eventos recibidos**, con identificador, tipo y contenido en orden cronológico. Ahí se comprueba que la reconexión de la 5.9.3 funcionó: **tras reconectar deben aparecer los eventos que se habían perdido**.

FIGURA 5.6

captura

Captura de la pestaña de eventos de una conexión SSE, con identificador, tipo y contenido en orden. Idealmente tomada después de una reconexión, para que se vean los eventos recuperados

Figura 5.6. El flujo de eventos en el panel de red

EXPERIMENTO — hacé tangible RN-F11

Conviene hacerlo sobre el TPI apenas tengas el canal andando.

1. Abrí la vista de pedidos con el canal conectado y verificá que los eventos llegan.
2. Activá el modo **sin conexión** en el panel de red.
3. Desde otra pestaña o con `curl`, hacé avanzar un pedido de estado.
4. Volvé a la vista. **Sigue mostrando el estado viejo, y no hay ningún error.**
5. Desactivá el modo sin conexión y mirá cuánto tarda en actualizarse.

Ese paso 4 es el punto: **no hay mensaje, no hay ícono rojo, no hay nada**; hay una pantalla que se ve normal y está mintiendo. Imagínatela en la cocina, con el cocinero mirándola para saber qué preparar. Por eso RN-F11 no es opcional.

5.11 — Seguridad y evolución

Todo lo que llega al cliente es visible. El token, las respuestas, la lógica de las guardas. Es la base de RN-F04 y **no tiene solución técnica del lado del cliente**, porque el código se ejecuta en una máquina que el atacante controla.

La cancelación es también una cuestión de corrección. Una petición que se resuelve después de que su vista desapareció puede **escribir sobre estado que ya no corresponde**: eso es un bug, no un desperdicio. El `AbortController` de la 5.5.4 evita esa clase de error además de la fuga de RN-F01.

El límite de intentos es una defensa compartida. El servidor lo impone, pero un cliente que respeta `Retry-After` **es parte de la defensa**; uno que no, es parte del problema, aunque sea de buena fe.

Y tres incorporaciones recientes resuelven cosas que hasta hace poco exigían código propio:

Novedad	Qué reemplaza	Por qué importa
<code>AbortSignal.timeout()</code>	El plazo de espera armado a mano	Menos código para algo que toda petición necesita (5.5.4)
<code>Array.fromAsync()</code>	Los bucles manuales sobre iterables asíncronos	Recorrer flujos con la comodidad de un arreglo
La interfaz de flujos	Esperar la respuesta completa en memoria	Procesar una respuesta grande a medida que llega

Una observación final: un canal con recuperación por identificador, sincronización inicial y recarga de respaldo es **la misma estrategia que usa cualquier sistema distribuido serio** —no confiar en la entrega, sino diseñar para que la pérdida sea recuperable—. **Lo que se aprende acá no es una particularidad del TPI.**

5.12 — Verificación: el checklist honesto

Nueve comprobaciones. **No son ejercicios: son el criterio para saber si el capítulo se entendió.**

- Predecir el orden de salida de un fragmento con código síncrono, `setTimeout`, promesas y `await`, y verificarlo. (5.3.1)
- Pedir un endpoint inexistente y **comprobar que el `catch` no se ejecuta**; corregirlo con `respuesta.ok`. (5.5.3)
- Convertir dos `await` independientes en un `Promise.all` y **medir la diferencia** con la limitación de red activada. (5.3.3 y 5.4.2)
- Provocar un error de CORS y explicar, mirando el panel de red, **si la petición llegó al servidor**. (5.6.2)
- Cancelar una petición con `AbortController` y verificar en el panel de red que figura como cancelada. (5.5.4)
- Distinguir en el código un fallo de red, un `500` y un `409`, con **un mensaje distinto para cada uno**. (5.7.1)

- Simular un 429 y verificar que el cliente **respete el Retry-After**. (5.7.3)
- Abrir un canal, cortar la conexión, provocar un cambio y **verificar tras reconectar que el evento perdido se recupera**. (5.9.3)
- Explicar por qué RN-F09 exige invalidar en vez de escribir, **con un ejemplo de estado inconsistente concreto**. (5.9.4)

5.13 — Los doce errores frecuentes

Todos tienen algo en común: **en el momento, no parecen errores**. Por eso son frecuentes.

El error	Por qué duele	Sección
Crear que <code>fetch</code> falla ante un 500	La promesa se cumple : sin <code>if (!respuesta.ok)</code> el cuerpo del error pasa por resultado válido	5.5.3
Encadenar <code>await</code> de peticiones independientes	Duplica el tiempo de carga sin razón, y se ve prolijo	5.4.2
Usar <code>forEach</code> con una función <code>async</code>	No espera nada: el usuario ve un éxito que no ocurrió	5.4.2
Elegir <code>Promise.all</code> cuando corresponde <code>allSettled</code>	Un fallo descarta todos los resultados, incluidos los buenos	5.3.3
Mostrar «error de conexión» ante un 4xx	El servidor respondió, con información que el usuario necesita	5.7.1
Comparar el texto del mensaje en vez del código	Una traducción rompe tu <code>if</code> sin dejar ningún error visible	5.7.2
Intentar arreglar CORS desde el frontend	No se puede: el complemento lo desactiva sólo en la máquina de quien lo instaló	5.6.2
Omitir el plazo de espera	<code>fetch</code> no tiene uno propio: la petición queda pendiente durante minutos	5.5.4
No cancelar al desmontar	Escrituras sobre estado obsoleto y la fuga del Capítulo 4. Viola RN-F01	5.5.4
Reintentar de inmediato ante un 429	Empeora lo que el límite contiene y, como cuenta por red, afecta a todos	5.7.3
Escribir en la caché con el contenido del evento	El estado no corresponde a ningún momento real del servidor. Viola RN-F09	5.9.4
Abrir un canal de eventos por vista	Consume el límite de conexiones y contradice al servidor. Viola RN-F10	5.9.4
Confiar en que el evento siempre llega	Datos viejos sin ningún error visible . Viola RN-F11	5.9.4

5.14 — Las actividades, y qué busca cada una

1. Predicción de orden

Dado un fragmento con `setTimeout`, tres promesas y dos `await`, escribir el orden de salida **antes** de ejecutarlo y explicar cada diferencia con lo observado, según el modelo de la sección 3.10.2.

Qué busca: *que compruebes que el orden de ejecución no es el de escritura, y que el modelo del Capítulo 3 lo predice exactamente.*

2. Capa `api/` mínima

Escribir `pedir(ruta, opciones)`: que agregue el token, verifique `respuesta.ok`, traduzca el código del catálogo del TPI —sección 14.1— a un error propio y aplique plazo de espera. Usarla en tres endpoints y verificar que **ninguna vista repite esa lógica**.

Qué busca: *que sientas la diferencia entre escribir la comprobación una vez y escribirla tres.*

3. Serie contra paralelo

Cargar un panel de cuatro estadísticas **de las dos formas**, medir ambas con la limitación de red en tres megabits y justificar cuál combinador corresponde.

Qué busca: *el número. Leer «es más lento» no convence a nadie; verlo, sí.*

4. Las tres categorías

Una vista que muestre un mensaje distinto ante un fallo de red, un `500` y un `409` por falta de stock. Provocar los tres casos y documentar **cómo se distingue cada uno en el código**.

Qué busca: *que el manejo de errores deje de ser un `catch` que dice «hubo un problema».*

5. Cancelación en una búsqueda

Un buscador que cancele la petición anterior en cada tecla. Verificar en el panel de red que aparecen canceladas y explicar **qué problema evita además del desperdicio**.

Qué busca: *que descubras el problema de las respuestas desordenadas, que es el que de verdad rompe la pantalla.*

6. Exploración: el hueco de la reconexión

Con el canal abierto, cortar la conexión, provocar **tres** cambios de estado desde otra pestaña y reconectar. Documentar qué eventos se recuperan y cuáles no, y relacionarlo con la **sección 11.3 del TPI** y con RN-F11. *(Requiere el backend del TPI en ejecución.)*

Qué busca: *que veas el hueco con tus ojos. Después de eso, RN-F11 no hay que justificarla.*

7. Exploración: qué protege CORS

Emitir la misma petición desde una página de otro origen y desde `curl`. Documentar cuál se bloquea y cuál no, y explicar **a quién protege** la política de mismo origen y por qué el servidor debe autorizar igual. *(Requiere `curl` y un servidor local.)*

Qué busca: *que quede claro que CORS vive en el navegador, y que el servidor no puede delegarle ni un gramo de su seguridad.*

5.15 — Síntesis: las once frases

1. La petición en segundo plano nació para que **Outlook Web Access se pareciera a un cliente de escritorio**, existió seis años sin nombre y se volvió ubicua cuando un ensayo de 2005 la nombró.
2. **Asincronía no es concurrencia**. Sigue habiendo un solo hilo: `await` no espera, **cede el control y vuelve**. Una `async` con un cálculo pesado adentro bloquea igual.
3. Las promesas resolvieron **el manejo de errores, no la indentación**: un solo `catch` cubre toda la cadena, con una idea sola: **el resultado futuro es un valor**.
4. **fetch no rechaza ante un error HTTP**. Un 404 o un 500 cumplen la promesa, porque la operación de red salió bien; sin respuesta `.ok`, el cuerpo del error pasa por resultado válido.
5. Encadenar `await` de peticiones independientes **multiplica el tiempo de carga sin motivo**, y `forEach` no espera promesas.
6. **CORS no protege al servidor: protege al usuario** de que un sitio lea respuestas de otro con sus credenciales. Se arregla en el servidor y sólo ahí, y el error lo emite el navegador después de que la respuesta llegó.
7. Red, protocolo y aplicación son **tres categorías distintas** de error. Un 409 por falta de stock es información de negocio, y se compara **por código**, nunca por el texto del mensaje.
8. **Toda petición lleva plazo de espera**, porque la peor falla no es la que falla: es la que queda colgada, sin error, con el usuario apretando el botón otra vez.
9. **Todo lo que llega al cliente es visible**. Las guardas de ruta son usabilidad —RN-F04— y ninguna forma de guardar el token es segura si hay un XSS: la defensa real es RN-F02.
10. El TPI eligió SSE por cinco criterios, y el decisivo es que **SSE es HTTP**. El mecanismo de publicación **no persiste los mensajes**, y de ahí salen las tres reglas: invalidar en vez de escribir (RN-F09), una sola conexión (RN-F10) y no depender del canal (RN-F11).
11. El canal es una **optimización de latencia, no la fuente de verdad**. Una interfaz que sólo se actualiza por eventos puede quedarse mostrando datos viejos **sin ningún error visible**, que es la peor forma de fallar.

5.16 — Qué leer, y en qué orden

Si lees una sola cosa

Archibald, *Tasks, microtasks, queues and schedules* (2015): la explicación más precisa del orden de ejecución de la sección 5.3.1, y **se puede ejecutar mientras se lee**.

Si lees tres

- **Beyer y otros**, *Site Reliability Engineering* (O'Reilly, 2016 — libre en sre.google/books), el capítulo sobre reintentos y espera creciente: explica **por qué la variación aleatoria de la sección 5.7.3 no es un detalle**.
- **MDN** sobre CORS y la **OWASP HTML5 Security Cheat Sheet**: cubren los casos que la especificación deja implícitos, que son los que te vas a encontrar.
- **Garrett**, *Ajax: A New Approach to Web Applications* (Adaptive Path, 2005): **se lee en veinte minutos** y muestra qué se consideraba imposible antes de la técnica.

Las fuentes normativas (para consultar, no para leer de corrido)

- **Fetch Standard** del WHATWG, en fetch.spec.whatwg.org: un **estándar viviente**, no una RFC. Define la interfaz de la 5.5, el modelo de respuestas y el intercambio de recursos entre orígenes de la 5.6, **incluida la lista de condiciones que evitan la verificación previa**.
- **HTML Living Standard**, su sección sobre `EventSource`: el formato del flujo de la 5.9.2, el reenvío del último identificador y la reconexión.
- **ECMA-262**: las promesas y `async/await`. Ojo con la distinción que ya señaló el Capítulo 3: **el bucle de eventos que determina el orden no está acá**, está en el estándar HTML.

- **RFC 6454** del IETF: la política de mismo origen de la 5.6.1. Y **RFC 9110**: los códigos de estado, la misma norma del Capítulo 1.

Y del TPI, cuatro secciones para tener abiertas

- **11.1** — los cinco criterios de la elección de SSE (5.9.1).
- **11.3** — el hueco de la reconexión y su recuperación (5.9.3).
- **11.4** — la autorización del canal, la multiplexación y la concesión del token en la cadena de consulta (5.8 y 5.9.4).
- **14.1** — el catálogo de errores que la capa `api/` traduce, con la **14.2**, que dice dónde (5.7.2).

Cierre: las siete cosas que hay que recordar

LAS SIETE

1. **Hay un solo hilo.** `await` no espera: **cede el control y vuelve.**
2. **La promesa es un valor, no una operación.** Por eso se puede guardar, pasar y combinar — y por eso **no se puede cancelar**: se cancela la petición de abajo.
3. **Si no dependen, van juntas.** Dos `await` seguidos de peticiones independientes son el doble de tiempo regalado. Y `forEach` no espera nada.
4. **`fetch` cumple con un 500.** El cartero entregó la carta; que traiga una mala noticia no es problema del correo. **Si no ves un `if (!respuesta.ok)`, el código está mal.**
5. **CORS protege al usuario, no al servidor,** vive en el navegador y se arregla del otro lado. Un `curl` ni se entera de que existe.
6. **Toda petición lleva plazo de espera,** y toda petición que puede quedar huérfana lleva `AbortController`: es RN-F01 con otra ropa.
7. **El canal de eventos es una optimización, no la verdad.** El evento es el timbre, no el paquete (RN-F09); uno solo por sesión (RN-F10); y la pantalla se actualiza igual si el canal se cae (RN-F11).

Y una octava, que no está escrita en el capítulo pero está en todas sus páginas: **la peor falla no es la que falla.** Un error explícito te dice dónde mirar; lo que arruina un sistema es el botón que gira para siempre, la lista que muestra resultados de otra búsqueda y el reloj de la estación que se paró marcando una hora creíble. Todo este capítulo es una sola disciplina: **hacer que las fallas se vean.**

Continúa en: Capítulo 6 — TypeScript: tipos sobre JavaScript y el contrato de la API, donde las respuestas que este capítulo trae sin forma conocida adquieren una, y donde el `total` que viaja como cadena decimal —RN-F08— encuentra su tipo correcto.